Game Report Summary

Name: Hatice Tunç
Student ID: 202340081
Date: 18-04-2025

Video Link: https://youtu.be/UR04mX8aS6s

Java Game Analysis Report

Introduction

The provided code implements a 2D platform game where players control an elephant character through multiple stages, each with unique challenges. The game consists of five stages with different mechanics:

1. **Stage 0**: Standard controls using arrow keys
2. **Stage 1**: Reversed left/right controls
3. **Stage 2**: Character jumps continuously with bouncy physics
4. **Stage 3**: Requires pressing a button 5 times to open the exit door
5. **Stage 4**: Changes control scheme to WAD keys

The objective in each stage is to navigate through obstacles and spikes, press buttons to open doors, and reach the exit pipe. The game tracks deaths, time elapsed, and includes reset functionality.

Implementation Details

The game is implemented using five Java classes that work together to create the complete gaming experience.

1. Main Class

The Main class serves as the entry point for the game. It creates five Stage objects with different parameters and adds them to an ArrayList, which is passed to the Game constructor. Each stage is configured with specific gravity, velocity, control keys, and hint messages.

Code Block 1:

```
public static void main(String[] args) {
    ArrayList<Stage> stages = new ArrayList<>();
    // Create stages with different parameters
    Stage stage0 = new Stage(-0.45, 3.65, 10, 0, 39, 37, 38,
```

```
            "Arrow keys are required", "Arrow keys move player, press button and enter the
second pipe");
    // ... more stages
    stages.add(stage0);
    // ... add more stages
    Game game = new Game(stages);
    game.play();
}
```

As shown in Code Block 1, each stage is initialized with distinct parameters that determine its physics and gameplay mechanics.

2. Stage Class

The Stage class defines the properties of each game level:

- Physics parameters (gravity, velocity)
- Control key mappings
- Stage number
- Clue and help text
- Random color generation for obstacles

Code Block 2:

```
public Stage(double gravity, double velocityX, double velocityY, int stageNumber,
             int rightCode, int leftCode, int upCode, String clue, String help) {
    this.gravity = gravity;
    this.velocityX = velocityX;
    // ... other initializations
}
```

The class also includes a method to generate random colors for obstacles, making each stage visually distinct, as shown in Code Block 2.

3. Player Class

The Player class represents the elephant character and handles:

- Position tracking (x, y coordinates)
- Movement and physics (velocity, gravity)
- Collision detection with:
    - Obstacles
    - Exit pipes
    - Buttons
    - Spikes
- Character respawning
- Drawing the character with appropriate sprites based on direction

Code Block 3:

```
public void move(Direction way) {
    if (way == Direction.RIGHT && (!checkCollision((getX() + stage.getVelocityX()),
getY()))) {
        setX((getX() + stage.getVelocityX()));
    }
    // ... other movement code
}
```

Code Block 4:

```
public void move() {
    if (stage.getStageNumber() == 2) {
        velocityY += stage.getGravity();
        double nextY = getY() + velocityY;

        if (!checkCollision(getX(), nextY)) {
            // No collision, continue movement
            setY(nextY);
        } else {
            // Collision detected, bounce properly
            // When we hit something, invert the velocity
            velocityY = -velocityY; // rebounce
            // Apply a small displacement to prevent getting stuck in collision
            if (velocityY > 0) {
                setY(getY() + 1); // Moving slightly up
            } else {
                setY(getY() - 1); // Moving slightly down
            }
        }
    } else {
        velocityY += stage.getGravity();
        if (!(checkCollision(getX(), getY() + velocityY))) {
            setY(getY() + velocityY);
        } else {
            velocityY = 0;
        }
    }
}
```

The player movement is handled by equations for physics simulation. In Code Block 4, we can see several important equations:

Equation 1:

```
velocityY += stage.getGravity()
```

Eq. 1 represents the acceleration due to gravity affecting the player's vertical velocity.

Equation 2:

```
nextY = getY() + velocityY
```

Equation 2 calculates the next Y position based on current position and velocity.

Equation 3:

```
velocityY = -velocityY
```

Equation 3 models the bounce physics by inverting the velocity upon collision, creating a realistic bounce effect.

4. Map Class

The Map class manages the game environment:

- Static obstacle definitions
- Door/button mechanics
- Pipe locations (start and exit)
- Spike hazards
- Drawing the environment
- Collision checking for the door

Code Block 5:

```
public void draw() {
    StdDraw.setPenColor(stage.getColor());
    for (int[] obstacle : obstacles) {
        int x1 = obstacle[0];
        int y1 = obstacle[1];
        int x2 = obstacle[2];
        int y2 = obstacle[3];
        double centerX = (x1 + x2) / 2.0;
        double centerY = (y1 + y2) / 2.0;
        double halfWidth = Math.abs(x2 - x1) / 2.0;
        double halfHeight = Math.abs(y2 - y1) / 2.0;
        StdDraw.filledRectangle(centerX, centerY, halfWidth, halfHeight);
    }
```

```
    // ... draw other map elements
}
```

In Code Block 5, we can see how the Map class calculates the center points and dimensions for drawing rectangular obstacles.

Equation 4:

```
centerX = (x1 + x2) / 2.0
centerY = (y1 + y2) / 2.0
```

Equation 4 calculates the center point of each obstacle.

Equation 5:

```
halfWidth = Math.abs(x2 - x1) / 2.0
halfHeight = Math.abs(y2 - y1) / 2.0
```

Equation 5 calculates the half-width and half-height needed for the StdDraw.filledRectangle method.

5. Game Class

The Game class is the core component that ties everything together:

- Game loop and state management
- Input handling
- Stage transitions
- UI elements (timer, death counter, help text)
- Game restart functionality
- End-of-game congratulations screen

Code Block 6:

```
public void play() {
    StdDraw.setCanvasSize(800, 600);
    StdDraw.setXscale(0, 800);
    StdDraw.setYscale(0, 600);
    StdDraw.enableDoubleBuffering();
    while(true){
        String currentTime = getFormattedElapsedTime(System.currentTimeMillis(),
stageStartTime, totalPauseTime);
        map.setIsDoorOpen();
        map.doorFalling();
        handleInput(currentStage);
        handleMouseClick();
        // ... other game logic
```

```
        draw();
        StdDraw.show();
        StdDraw.pause(16);  // ~60 FPS
        StdDraw.clear();
    }
}
```

Code Block 7:

```
private String getFormattedElapsedTime(long x2, long x1, long x3) {

    long elapsed = x2 - x1 - x3;

    long minutes = (elapsed / 60000);
    long seconds = (elapsed % 60000) / 1000;
    long millis = (elapsed % 1000) / 10; // two-digit ms for aesthetics

    return String.format("%02d : %02d : %02d", minutes, seconds, millis);
}
```

The Game class implements a timer that shows minutes, seconds, and milliseconds as shown in Code Block 7. The time calculations use the following equations:

Equation 6:

```
elapsed = x2 - x1 - x3
```

Equation 6 calculates the elapsed time by subtracting the start time (x1) and total pause time (x3) from the current time (x2).

Equation 7:

```
minutes = elapsed / 60000
seconds = (elapsed % 60000) / 1000
millis = (elapsed % 1000) / 10
```

Equation 7 converts the elapsed milliseconds into minutes, seconds, and centiseconds for display.

Interaction Between Classes

The classes interact in the following ways:

1. **Game-Stage Interaction**: The Game class retrieves stage information to set up physics parameters and control schemes as seen in Code Block 6.
2. **Game-Player-Map Interaction**: The Game class creates Player and Map objects for each stage. It passes user input to the Player for movement and checks with the Map for collisions.

3. **Player-Map Interaction**: The Player uses Map's static methods to check for collisions with obstacles, spikes, buttons, and the exit pipe, as demonstrated in Code Block 3 and Code Block 4.
4. **Map-Stage Interaction**: The Map uses Stage's color for drawing obstacles and adapts to stage-specific mechanics, as shown in Code Block 5.

The game is built using five main classes that work together. The Main class starts the game by creating the levels (as Stage objects) and passing them to the Game class. The Game class is in charge of running the game: it listens to keyboard input, updates the game screen, handles timing, resets, and moves the player from one level to the next.

Each level is defined by a Stage object, which contains settings like gravity, movement speed, and key bindings. The Game class creates a Player and a Map based on the current stage. The Player class controls the elephant's movement, jumping, and collision with spikes and buttons. The Map class draws the environment and checks for things like spike collisions and whether the door should open. All these classes communicate with each other through shared data like the current stage, position of the player, and game state.

Special Game Mechanics

The game implements several interesting mechanics:

1. **Button and Door System**: When the player stands on a button, it triggers the door to open. In Stage 3, the button must be pressed 5 times, as implemented in Map class.
2. **Death and Respawn**: When the player touches spikes, they respawn at the starting position, and the death counter increases, as handled in the Game class.
3. **Physics**: Each stage has unique physics parameters, with Stage 2 featuring bouncy mechanics as shown in Code Block 4 and Equation 3.
4. **Control Schemes**: Different stages use different control mappings, adding to the challenge, as configured in Code Block 1.

The StdDraw library is used for all graphics rendering, displaying the game at approximately 60 frames per second (as shown by the 16ms pause in Code Block 6). The game also uses simple sprite images for the elephant character and spike hazards.